

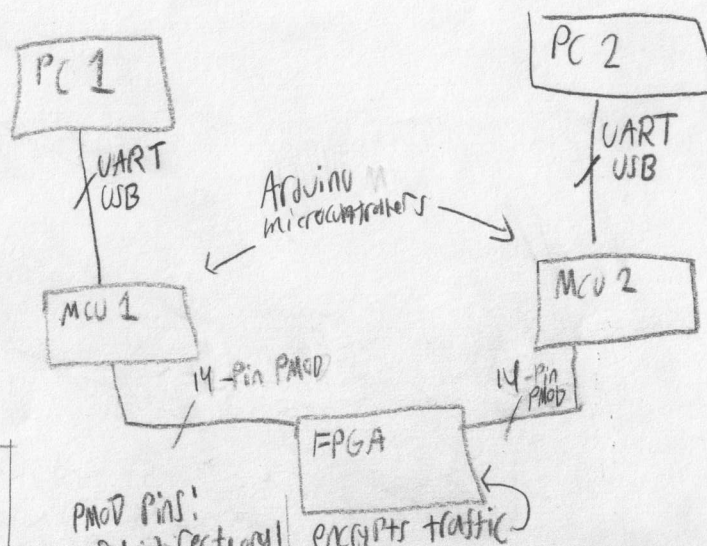
- FPGA sends 8 bits combinatorially and offloads UART to the MCU
- Probably data should still be in hex and not binary
- much slower. Probably like 15 KiB/s max
- FPGA has two parallel instances of whatever AES mode it is using instead of one

thoughts:

This design is not exactly simpler than using Ethernet, but it is more familiar complexity. Overall, I don't like this solution, and it feels hacky. More devices to program and power. Also, I don't have two of the same MCUs, so I would have to restrict the program to what works on both the mega 2560 and the uno.

I'm not using the term "PMOD" correctly but idc. It is breadboard wires.

board's reference designators J13 and J14



PMOD Pins:

- 8 bidirectional data lines
- master req
- slave ack
- slave req
- master ack 1&2
- bus owner

master doesn't need both req pins

Something similar to the

- FPGA ↔ MCU will use 4-Phase bus handshake, plus extra rules to ensure no data races.
- FPGA is master and MCU is slave.
- MCUs are basically just a passthrough mechanism since the FPGA only has one RX/TX pair.

MCUs use 2 256-byte ring buffer FIFOs to avoid data loss, 256 to avoid software math emulation for 8-bit AVR Arduino CPUs, one buffer will be used for each direction

1. Sender raises req 1
2. master gives bus ownership to sender
3. sender writes to data bus
4. sender raises req 2
5. receiver reads from the data bus
6. receiver raises ack
7. sender stops driving data bus
8. sender lowers req 1&2
9. receiver lowers ack
10. master resets ownership

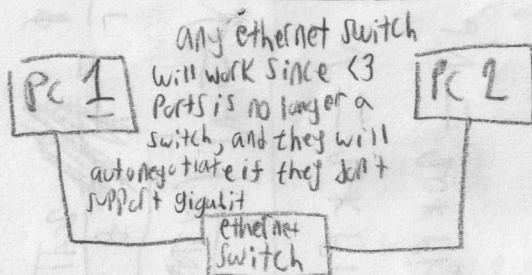
- step 10 happens in parallel to either 8 or 9, depending on who the master is
- if the sender is the master, then steps 1-4 can all happen in parallel. In this case, req 1 and 2 are just one pin.

Design 2

- All connections are ethernet.
- FPGA has RGMII gigabit ethernet.
- Ethernet switch just redirects to the right port (not a router)
- FPGA will be given a static MAC somewhere in 02:xx:xx:xx:xx:xx.
- Raw packets instead of full TCP/IP since that is too complicated for the purpose
- PCs should be Linux instead of Windows for the simpler raw socket API (ARP off too)

Discovery Sequence:

1. PC1 sends hello to FPGA (first one is PC1)
 2. FPGA sends acknowledge to PC1.
 3. if PC1 doesn't see ack, goto Step 1
 4. PC2 sends hello to FPGA
 5. FPGA sends ack to PC2
 6. if PC2 doesn't see ack, goto Step 4.
 7. FPGA sends start to both PCs; link is up
- Parts 1 & 2 can mostly be in parallel. The FPGA will assign PC1 or PC2 basically arbitrarily since it doesn't actually matter.
 - Part 3 always happens last



RTL 8211F-CG PHY

H82401S transformer
no NIC

Key establishment: undecided between Pre-Shared and ECDH. ECDH would require a 256-bit curve to have the same 128-bit security as AES-256.

- Perhaps curve25519? Using ECDH over PSK will add complexity, but also allow dynamic key re-establishment without starting over and manually re-entering keys.
- I chose ECDH over DH for the smaller key shares to get the same security. This requires less complexity on the FPGA side for the message transfers.

- I have not thoroughly considered retransmission for key-establishment, probably a message counter like how TLS does it. Then perhaps a command for the FPGA to re-request specific blocks if it.

The maximum data payload is 32 bytes (key), which leaves 14 bytes for extra metadata fields. The EtherType will probably be 0x88B5 or 0x88B6, which are reserved for local/experimental stuff.

If I decide to implement packet retry, then this will be the process:

1. Sender sends message with retry = false
2. Receiver sends back "ok"
3. If sender doesn't see "ok", it assumes its message's packet dropped and resends the message with retry = true.

Preamble, SFD, and IFG are handled by the PHY at the hardware layer. I can ignore them

- Ethernet requires a CRC32 sum of the payload, which is not handled by the PHY. and MACs
- Ethernet switches drop packets with CRC mismatches, so probably I don't need to implement CRC checks unless I don't trust the switch
- Packet timeouts will probably be 20ms - 1ms. Perhaps 150% of the expected round trip time?
- FPGA will have the encryption be entirely in parallel since ethernet RX and TX can fire in parallel, so there is essentially no benefit to arbitrarily introducing stalls
- Probably I won't support > 2 connected PCs
- No keepalive will be required. The FPGA will assume both sides are connected. There will probably be a reset command for when a PC reconnects after a hard disconnect. or perhaps also a soft vs hard reset system with reset, stop, and resume commands or something.
- Gowin has IP cores for DDR, so I won't have to manually use a rising and a falling-edge flipflop to handle the DDR transfers on the FPGA. There are also a few deprecated DDR IP cores

a lot less moving parts than design 1, but each part is doing a lot more.

Since ethernet packet drops are so low, I might just skip implementing all the retransmission stuff.